

Carte Avansata

Tehnici Machine Learning - Nivel Expert

Manual aprofundat pentru Senior Data Scientist si ML Engineer.

Matematica, algoritmi, capcane si decizii de productie - pentru toate cele 12 categorii ML

Diana Nenu | Lucrare Disertatie | Master Data Science and AI
Bucuresti, 2026 | Volumul II

CUPRINS

Introducere	3
Categoria 1 (avansat). Data Cleaning	4
Categoria 2 (avansat). Exploratory Data Analysis	10
Categoria 3 (avansat). Feature Engineering	14
Categoria 4 (avansat). Feature Selection	19
Categoria 5 (avansat). Data Splitting si Validation	22
Categoria 6 (avansat). Modele de Machine Learning	26
Categoria 7 (avansat). Hyperparameter Tuning	33
Categoria 8 (avansat). Metrici de Evaluaare	37
Categoria 9 (avansat). Explainability	41
Categoria 10 (avansat). Deployment	45
Categoria 11 (avansat). Monitoring	49
Categoria 12 (avansat). Tehnici Emergente	53
Roadmap pentru a deveni expert ML	58
Resurse recomandate de invatare	60
Concluzie	62

Introducere

Acest manual este al doilea volum din seria Carte de Referinta pentru Tehnici Machine Learning. Spre deosebire de primul volum, care prezinta tehnicile la nivel general (cum functioneaza, cand se folosesc), acest volum coboara la nivel expert: matematica completa din spate, algoritmi pas-cu-pas, capcane subtile, decizii de productie, comparatii detaliate si cod rulabil de productie.

Documentul este destinat cititorilor care au inteles deja conceptele fundamentale si vor sa atinga nivelul de expert care este cerut in pozitiile de Senior Data Scientist si ML Engineer. Fiecare categorie aprofundeaza tehnicile cele mai folosite, cu accent pe ce face diferenta intre un practicant junior si unul senior.

Folosirea recomandata: parcurgi fiecare categorie complet, apoi te intorci la cele care iti sunt mai putin clare. La final ai un nivel suficient pentru a sustine un interviu tehnic si pentru a discuta in profunzime cu echipa de inginerie.

Categoria 1 (avansat). Data Cleaning

1.1. De ce sunt date lipsa o problema fundamentala

Algoritmii ML clasici (linear regression, KNN, SVM, rețele neurale) nu pot procesa NaN matematic. O singura valoare lipsa într-o coloană înseamnă că tot rândul este inutilizabil pentru acest gen de modele. Algoritmii bazati pe arbori (XGBoost, LightGBM) au tratare nativa a NaN - dar e bun de stiut, nu o garantie. In productie, decizia de imputare e adesea cea care diferentiaza un model robust de unul fragil.

O greseala frecventa este sa imputezi orbeste, fara a intelege de ce sunt datele lipsa. Daca pattern-ul de NaN are o cauza sistematica (de exemplu, anumiti utilizatori nu au declarat venitul), imputarea cu media introduce bias semnificativ. Inainte de orice imputare, trebuie sa intelegi natura datelor lipsa.

1.2. Tipuri de date lipsa (clasificarea Rubin)

Donald Rubin (1976) a propus o clasificare statistica a tipurilor de date lipsa care e fundamentala pentru orice profesionist al datelor. Cunoasterea acestei taxonomii e standardul in cercetarea aplicata.

MCAR (Missing Completely At Random)

Probabilitatea de a fi NaN nu depinde de nimic. Un senzor a cazut accidental, un utilizator a inchis browser-ul accidental, o conexiune de retea s-a intrerupt. Datele lipsa sunt complet aleatoare.

Test statistic: Little's MCAR test (statsmodels). Daca $p\text{-value} > 0.05$, nu putem respinge ipoteza MCAR. In practica, MCAR e rar in date reale - cele mai multe NaN au cauze.

Solutie: orice metoda functioneaza (drop, imputare cu media). Nu introduci bias indiferent ce alegi.

MAR (Missing At Random)

Probabilitatea de NaN depinde de alte variabile observate. De exemplu, NaN la salariu pentru toti angajatii din departamentul X (poate ei nu au declarat). Cunoscand departamentul, pot prezice probabilitatea de NaN.

MAR este cel mai comun caz in practica. E ceea ce metodele moderne de imputare (KNN, MICE) sunt construite sa rezolve - folosesc corelatiile cu alte variabile.

Solutie: imputare avansata care foloseste alte features. Imputarea cu media e suboptima.

MNAR (Missing Not At Random)

Probabilitatea de NaN depinde de valoarea insesi. NaN la salariu doar pentru salariile foarte mari (oamenii cu venituri mari nu raspund). Sau NaN la satisfactie pentru clientii nemultumiti.

Cel mai dificil caz. Imputarea simpla introduce bias structural. Solutiile riguroase necesita modele explicite de selectie (Heckman correction, copula models).

In productie, MNAR se rezolva prin: 1) Colectare de date suplimentare; 2) Modele cu mecanism explicit de selectie; 3) Acceptare bias si documentare clara a limitarilor.

1.3. Diagnoza patternurilor de NaN

Inainte sa decizi metoda de imputare, vizualizezi pattern-urile cu missingno:

```
import missingno as msno
```

```
msno.matrix(df)          # heatmap NaN-uri pe randuri si coloane
```

```
msno.heatmap(df)      # corelatii NaN intre coloane
msno.dendrogram(df)   # grupari de coloane care au NaN-uri impreuna
```

Daca msno.heatmap arata corelatie inalta intre col_A si col_B (cand A e NaN, si B e NaN), e probabil MAR. Coloanele care apar grupate in dendrogram au probabil acelasi mecanism de NaN.

1.4. Workflow practic pentru tratarea NaN in productie

Pasul 1: Calculezi procentul de NaN pe fiecare coloana.

```
df.isna().mean().sort_values(ascending=False)
```

Pasul 2: Pentru coloane cu peste 60% NaN, drop coloana (informatie prea putin recuperabila).

Pasul 3: Pentru coloane cu 30-60% NaN, investigatie atenta. Posibil drop daca corelatia cu target e mica, posibil imputare cu indicator daca e mare.

Pasul 4: Pentru coloane cu 5-30% NaN, imputare cu metoda potrivita (KNN sau MICE pentru MAR).

Pasul 5: Pentru coloane cu sub 5% NaN, drop sau imputare simpla (mediana).

1.5. Imputare cu indicator (best practice)

Truc esential pe care multi data scientisti il ignora: cand imputezi, pastrezi si o coloana flag care zice 1 daca era NaN, 0 altfel. Modelul invata sa trateze diferit valorile imputate fata de cele reale.

```
df['col_was_missing'] = df['col'].isna().astype(int)
df['col'] = df['col'].fillna(df['col'].median())
```

Asta da modelului semnalul ca valoarea respectiva e o ghicitura, nu un fapt. Frecvent ignorat, dar uneori imbunatateste R^2 cu 3-5%. Pentru tree-based models, aceasta tehnica e foarte puternica.

1.6. KNN Imputer - capcana subtila la scaling

KNN calculeaza distante euclidiene. Daca o coloana are valori 0-1 si alta 0-1.000.000, distanta e dominata de a doua. Trebuie sa scalezi datele inainte sa imputezi.

```
from sklearn.preprocessing import StandardScaler
from sklearn.impute import KNNImputer

# Scalezi inainte, imputezi, des-scalezi
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)
imputer = KNNImputer(n_neighbors=5)
X_imputed = imputer.fit_transform(X_scaled)
df_final = scaler.inverse_transform(X_imputed)
```

Hyperparametri importanti la KNN Imputer: n_neighbors (default 5; mai mare = mai stabil dar mai putin specific), weights ('distance' = vecini mai apropiati conteaza mai mult).

1.7. Comparatie tehnica intre metodele de imputare

KNN Imputer: lent pe seturi mari (millioane randuri), bun pe corelari liniare, sensibil la scaling. Default in pipeline rapid.

MICE: foarte lent, excelent pentru MAR cu corelari complexe, convergenta nu garantata. Standard de aur in cercetare medicala.

MissForest: mediu ca viteza, excelent pentru patterns neliniare, foloseste Random Forest iterativ. Lent pe milioane randuri.

In productie practic: KNN (k=5, weighted) e default pentru proiecte rapide. MICE doar in proiecte unde calitatea conteaza enorm si timpul nu e o problema.

1.8. Tratarea outliers - clasificare expert

Un outlier nu este intotdeauna o eroare. Trebuie sa distingem trei categorii distincte care necesita tratamente diferite.

Errori de masurare

Senzor defect, transcriere gresita, glitch in sistem. Aceste outliers sunt zgomot si trebuie eliminate sau corectate. Exemplu: temperatura corpului inregistrata 250 grade Celsius (clearly impossible).

Outliers contextuali

O valoare care e normala in alt context dar anormala aici. Temperatura de 30 grade Celsius in Bucuresti vara e normala, in Stockholm iarna e anomalie. Solutia: detectie cu features contextuale (coordonate, sezon, hour-of-day).

Evenimente extreme reale

Flash crash, criza energetica, pandemii, varfuri Black Friday. Sunt outliers statistici dar evenimente reale care contin informatie. Solutia: pastreaza dar marcheaza cu flag pentru analize separate.

1.9. IQR method - logica matematica completa

Boxplot-ul lui Tukey (1977) propune $1.5 \cdot \text{IQR}$ ca prag standard. De ce 1.5? Pentru o distributie normala, $Q3 + 1.5 \cdot \text{IQR}$ corespunde la aproximativ percentila 99.3%. Adica doar 0.7% din date ar fi marcate outlier doar prin sansa intr-o distributie perfect normala.

Cu $k=3$: probabilitate aproximativ 0.000003 (foarte conservator). Doar valorile extrem de rare.

Matematic: pentru distributie normala, IQR aproximativ $1.349 \cdot \text{std}$. Deci $Q3 + 1.5 \cdot \text{IQR}$ aproximativ $\text{mean} + 2.7 \cdot \text{std}$ (similar cu pragul de 3 sigma).

Cand IQR esueaza: distributii bimodale. Imagineaza-ti o distributie cu doua varfuri (de exemplu, vanzari foarte mari Black Friday + vanzari normale restul anului). IQR ar exclude tot Black Friday.

1.10. IQR pe sub-grupuri

Solutia pentru distributii bimodale: detectie pe sub-grupuri.

```
df['is_outlier'] = (
    df.groupby('season')['value']
        .transform(lambda x: (x < x.quantile(0.25) - 1.5*(x.quantile(0.75) - x.quantile(0.25))) |
                             (x > x.quantile(0.75) + 1.5*(x.quantile(0.75) - x.quantile(0.25))))
)
```

Asa, valori care sunt outlier in iarna pot fi normale vara. Aceasta abordare e standard in detectie anomalii contextuale.

1.11. Isolation Forest - matematica completa

Intuitia: punctele rare (outliers) sunt mai usor de izolat in arbori aleatori. Daca pentru un punct ai nevoie de doar 3 taieturi pentru a-l izola, vs 15 pentru un punct normal, primul e probabil outlier.

Algoritm pas-cu-pas: 1) Construisti N arbori binari, fiecare cu max 256 sub-esantion. 2) Pentru fiecare arbore: alegi feature aleator + prag aleator. Repeti pana cand fiecare frunza are 1 punct. 3) Pentru fiecare observatie X: calculezi adancimea medie $h(x)$ in toti arborii. 4) Anomaly score: $s(x, n) = 2^{(-h(x) / c(n))}$ unde $c(n)$ e o constanta de normalizare.

Score in $[0, 1]$: 0.5 = normal, aproape de 1 = anomalie sigura, aproape de 0 = inlier sigur. Avantaj cheie: complexitate $O(n \log n)$ - rapid chiar pe milioane randuri.

1.12. Cod complet de detectie cu vizualizare

```
import numpy as np
from sklearn.ensemble import IsolationForest
import matplotlib.pyplot as plt

iso = IsolationForest(
    n_estimators=100,
    contamination=0.05,
    max_samples=256,
    random_state=42,
    n_jobs=-1,
)
iso.fit(X)

scores = -iso.score_samples(X)
labels = iso.predict(X)

plt.hist(scores, bins=50)
plt.axvline(np.percentile(scores, 95), color='red', label='95th percentile')
plt.title('Distributie anomaly scores')
plt.legend()
```

Cand functioneaza prost: cand contaminarea e foarte inalta (peste 50% outliers - nu mai e clar ce e normal). Pentru fraud detection in finantate (contaminare 0.1-1%), Isolation Forest e standardul de facto.

Categoria 2 (avansat). Exploratory Data Analysis

2.1. Dincolo de describe() - analiza completa a distributiei

df.describe() ofera doar statistici de baza. Pentru o intelegere completa a unei distributii, ai nevoie de skewness si kurtosis, plus testarea normalitatii.

Skewness (asimetria)

Masura asimetriei distributiei. Formula: $E[(X - \mu)^3] / \sigma^3$.

skew = 0: distributia e simetrica (gaussian, Student t).

skew > 0: coada lunga la dreapta. Tipic pentru preturi, venituri, dimensiuni de fisiere.

skew < 0: coada lunga la stanga. Rar in date naturale (uneori scoruri de teste cu ceiling).

Regula: daca $|skew| > 1$, considera o transformare log sau Box-Cox inainte sa aplici modele liniare.

Kurtosis (turtirea)

Cat de ascutita sau plata e distributia fata de gaussiana. Formula: $E[(X - \mu)^4] / \sigma^4$.

kurt = 3 (sau excess kurtosis = 0): distributie normala.

kurt > 3 (excess > 0): distributie ascutita cu cozi grase (mai multi outliers). Tipic pentru rentabilitati financiare.

kurt < 3 (excess < 0): distributie plata. Mai rar in date naturale.

2.2. Test de normalitate - Shapiro-Wilk

Cand vrei sa decizi daca poti aplica teste parametrice (t-test, ANOVA) sau ai nevoie de non-parametrice (Mann-Whitney, Kruskal-Wallis), Shapiro-Wilk e standardul.

```
from scipy import stats
stat, p = stats.shapiro(df['col'])
# Daca p < 0.05, distributia NU e normala (rejecti H0)
```

Atentie: pentru seturi foarte mari (peste 5000 puncte), testul devine prea sensibil. Detecteaza diferente foarte mici fata de gaussiana care nu sunt practic relevante. Pentru date mari, foloseste vizualizari (Q-Q plot) sau Anderson-Darling.

2.3. Pearson - intelegere geometrica

Formula: $r = \frac{\sum((x_i - \text{mean}(x))(y_i - \text{mean}(y)))}{(n * \text{std}(x) * \text{std}(y))}$.

Geometric: e cosinusul unghiului dintre vectorii x si y dupa centrare. $r = 1$ inseamna vectori paraleli, $r = -1$ antiparaleli, $r = 0$ perpendiculari. Aceasta perspectiva geometrica te ajuta sa intelegi de ce Pearson masoara doar relatiile liniare.

Capcana clasica: poate fi 0 desi exista relatie clara.

```
x = np.linspace(-5, 5, 100)
y = x**2 # parabola perfecta
np.corrcoef(x, y) # ~ 0 desi y e complet determinat de x
```

Pearson detecteaza relatii liniare. Pentru parabolic, foloseste Spearman sau Mutual Information.

2.4. Spearman - logica completa

Algoritm: 1) Sortezi valorile lui x si asignezi ranguri (1, 2, 3, ...). 2) La fel pentru y. 3) Calculezi Pearson pe ranguri.

Avantaj: capteaza orice relatie monoton crescatoare (linie dreapta, exponentiala, logaritmica). Nu doar liniar.

Capcana: pierde informatie despre magnitudinea diferentelor. Doi puncte cu valori 1 si 2 au aceeaasi distanta in ranguri ca puncte cu 1 si 1000.

Cand sa preferi Spearman fata de Pearson: cand suspectezi outliers, cand relatia e monoton dar non-liniara, cand variabilele au distributii foarte diferite.

2.5. Mutual Information - definitia matematica

Definitie: $I(X; Y) = \sum_x \sum_y p(x,y) * \log(p(x,y) / (p(x) * p(y)))$.

Intuitiv: cat de mult scade incertitudinea despre Y daca cunosti X. $I(X; Y) = 0$ doar daca X si Y sunt complet independente.

Avantaj decisiv: detecteaza orice relatie - liniar, neliniar, ne-monotona, XOR. Pentru xor ($X=0, Y=0 \rightarrow 0$; $X=1, Y=1 \rightarrow 0$; $X=0, Y=1 \rightarrow 1$; $X=1, Y=0 \rightarrow 1$), Pearson si Spearman dau 0, dar Mutual Information detecteaza relatia perfecta.

Capcana: implementarea pentru variabile continue depinde de binning (cum impartii valorile in bin-uri). Foloseste implementarea sklearn (mutual_info_regression / mutual_info_classif) care foloseste estimari KDE robuste.

2.6. Workflow pentru detectarea relatiilor

Pasul 1: Calculeaza Pearson pe toate perechile - prima privire (rapid, $O(n^2)$).

Pasul 2: Daca observi Pearson aproximativ 0 dar suspectezi relatie - calculeaza Spearman pe acea pereche.

Pasul 3: Daca Spearman tot 0 dar inca suspectezi - Mutual Information.

Pasul 4: Daca MI tot 0 - probabil chiar nu exista relatie semnificativa.

Vizualizeaza scatter plot pentru a verifica vizual. Statisticile pot insela cand exista pattern-uri vizuale clare (clusters, valori).

2.7. Q-Q plot pentru normalitate

Quantile-Quantile plot: compari quantilele observate cu quantilele teoretice ale distributiei normale. Daca punctele cad aproximativ pe linia dreapta, distributia e aproape normala.

```
from scipy import stats
import matplotlib.pyplot as plt

stats.probplot(df['col'], dist='norm', plot=plt)
plt.title('Q-Q Plot pentru normalitate')
plt.show()
```

Q-Q plot e mai informativ decat un singur p-value. Vezi exact unde si cum diverge distributia (cozi grase? skew?).

Categoria 3 (avansat). Feature Engineering

3.1. Decizia One-Hot vs Target Encoding pe baza cardinalitatii

Cardinalitatea = numarul de categorii distincte. Decizia tehnica corecta:

Cardinalitate 2-10: One-Hot Encoding. Simplu, interpretabil, suficient.

Cardinalitate 10-50: One-Hot sau Target Encoding. Decizia depinde de marimea datasetului si de algoritm. Pentru tree-based, One-Hot e bun. Pentru linear models cu regularizare, Target.

Cardinalitate 50-1000: Target Encoding cu smoothing. One-Hot ar exploda dimensionalitatea.

Cardinalitate peste 1000: Hashing trick sau embeddings. Target Encoding inca functioneaza dar trebuie atent la leakage.

3.2. Target Encoding - capcana de leakage

Naive: pentru fiecare categorie c , calculezi $media_c = \text{mean}(y[X == c])$ pe TOT setul. Aceasta e cea mai mare capcana - introduce data leakage masiv.

Problema: la antrenare, modelul vede valori care includ target-ul pentru fiecare categorie. Modelul invata sa decoda target-ul prin features-ul encodat.

3.3. Solutia 1: smoothing

Formula: $\text{encoded_c} = (n_c * media_c + \text{smoothing} * media_globala) / (n_c + \text{smoothing})$.

Cand n_c (numar observatii pentru categoria c) e mic, encoding-ul e dominat de media globala (mai stabil). Cand e mare, e dominat de media specifica. Asta reduce leakage-ul si imbunatateste robustetea.

```
from category_encoders import TargetEncoder

te = TargetEncoder(smoothing=10) # smoothing=0 dezactiveaza
X_encoded = te.fit_transform(X, y)
```

3.4. Solutia 2: out-of-fold encoding (mai sigur)

Imparti datele in K folduri. Pentru fiecare fold, calculezi encoding-ul folosind doar celelalte K-1 folduri. Asta garanteaza zero leakage.

```
from sklearn.model_selection import KFold

encoded = np.zeros_like(y, dtype=float)
kf = KFold(n_splits=5)
for train_idx, val_idx in kf.split(X):
    means = y.iloc[train_idx].groupby(X.iloc[train_idx]).mean()
    encoded[val_idx] = X.iloc[val_idx].map(means)
```

Cand antrenez pe fold-ul 1-4, encoding-ul vine din 1-4. Cand antrenez pe 1-3+5, encoding-ul vine din 1-3+5. Niciodata fold-ul curent. Aceasta e tehnica standard in competitii Kaggle de top.

3.5. Multi-scale time features

In loc de un singur lag, foloseste medii pe scale multiple cu Exponential Weighted Moving Average. Pondere exponential descrescatoare pentru valori vechi - mai responsive decat rolling mean simplu.

```
df['target_ema_short'] = df['target'].ewm(span=3).mean()
```

```
df['target_ema_medium'] = df['target'].ewm(span=12).mean()  
df['target_ema_long'] = df['target'].ewm(span=48).mean()
```

EMA span = perioada efectiva. Span=3 inseamna ca ultimele 3 valori conteaza ~80% din pondere. Span=48 inseamna ca ultimele 48 conteaza ~80%.

3.6. Detrending si destemarea sezonality

Pentru ARIMA si modele liniare, stationarizezi seria - elimini trendul si sezonality pentru a obtine reziduuri.

```
from scipy.signal import detrend  
from statsmodels.tsa.seasonal import seasonal_decompose  
  
# 1. Detrending: scoti trendul  
df['target_detrended'] = detrend(df['target'])  
  
# 2. Decomposition: scoti sezonality  
result = seasonal_decompose(df['target'], model='additive', period=24)  
df['target_residual'] = result.resid  
df['target_trend'] = result.trend  
df['target_seasonal'] = result.seasonal
```

Pentru XGBoost / LSTM nu e necesar (modelele moderne invata trendul singure). Detrending e standard doar pentru ARIMA, modele liniare clasice si analiza statistica.

3.7. Lag features cu pondere temporală

In loc de lag_24 ca un singur numar, foloseste media ponderata a ultimelor N momente cu decay exponential.

```
weights = np.exp(-np.arange(168) / 24)  
weights = weights / weights.sum()  
df['target_decay_weighted'] = df['target'].rolling(168).apply(  
    lambda x: (x * weights).sum()  
)
```

Asta da pondere mai mare valorilor recente. Capteaza tendinta lenta dar prefera momentele recente. Util in domenii unde recentitatea conteaza (de exemplu, comportament client).

3.8. Interaction features

Pentru efecte combinate (cand efectul lui A depinde de B), creezi features de interactiune.

```
# Multiplicare  
df['temp_x_humidity'] = df['temperature'] * df['humidity']  
  
# Polynomial complete  
from sklearn.preprocessing import PolynomialFeatures  
poly = PolynomialFeatures(degree=2, interaction_only=True)  
X_poly = poly.fit_transform(X)
```

Pentru tree-based models, interactiunile sunt capturate automat (arborii fac splits combinate). Pentru modele liniare, trebuie sa le construisti manual.

Categoria 4 (avansat). Feature Selection

4.1. SHAP-based selection - workflow complet

```
import shap
import xgboost as xgb
import pandas as pd
import numpy as np

# 1. Antrenezi modelul
model = xgb.XGBRegressor(n_estimators=300)
model.fit(X_train, y_train)

# 2. Calculezi SHAP values
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_train)

# 3. Importanta = media absoluta a SHAP values per feature
importance = pd.DataFrame({
    'feature': X_train.columns,
    'mean_abs_shap': np.abs(shap_values).mean(axis=0),
}).sort_values('mean_abs_shap', ascending=False)

# 4. Selectezi top K
top_k = importance.head(20)['feature'].tolist()
X_train_reduced = X_train[top_k]
```

De ce SHAP > feature_importances_ standard: Tree feature_importances_ are bias spre features cu cardinalitate mare (mai multi splits posibili = mai mare importance fictiva). SHAP foloseste teoria jocurilor pentru atribuire echitabila bazata pe contributia reala la predictii.

4.2. Permutation Importance cu intervale de incredere

```
from sklearn.inspection import permutation_importance

result = permutation_importance(
    model, X_test, y_test,
    n_repeats=30,
    random_state=42,
    n_jobs=-1,
)

importance_df = pd.DataFrame({
    'feature': X_test.columns,
    'importance_mean': result.importances_mean,
    'importance_std': result.importances_std,
}).sort_values('importance_mean', ascending=False)
```

Cand intervalele de incredere se suprapun intre 2 features, nu poti spune ca unul e mai important decat altul. Multi data scientisti ignora aceasta nuanta. Pentru a fi rigorous, foloseste `n_repeats >= 30` si interpreteaza intervalele de incredere.

4.3. Boruta - statistical test rigorous

Algorithm Boruta: 1) Pentru fiecare feature real, creezi o copie shuffled (shadow feature). 2) Antrenezi modelul pe features reale + shadow. 3) Compari importanta featurelor reale cu max importanta a shadow features. 4) Daca un feature real e clar mai mare decat max shadow, e marcat important.

Repeti pentru N iteratii. Un feature e considerat important daca trece testul in majoritatea iteratiilor.

```

from boruta import BorutaPy
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(n_jobs=-1, max_depth=5)
boruta = BorutaPy(rf, n_estimators='auto', verbose=2, max_iter=100)
boruta.fit(X.values, y.values)

print('Confirmed:', X.columns[boruta.support_].tolist())
print('Tentative:', X.columns[boruta.support_weak_].tolist())

```

BorutaShap (versiune moderna): foloseste SHAP values in loc de feature_importances_. Mai robust dar mai lent.

4.4. Lasso path - selectie automata

```

from sklearn.linear_model import lasso_path
import numpy as np

alphas, coefs, _ = lasso_path(X, y, alphas=np.logspace(-4, 1, 50))

# Pentru fiecare alpha, vezi cati coeficienti sunt non-zero
n_features_per_alpha = (coefs != 0).sum(axis=0)

```

Lasso path arata cum se schimba coeficientii pe masura ce alpha (regularizarea) creste. Vezi care features 'cad' la zero primele - acelea sunt cele mai putin importante.

Categoria 5 (avansat). Data Splitting si Validation

5.1. Walk-forward validation - implementare riguroasa

Walk-forward simuleaza scenariul real de productie - antrenezi pe trecut, prezici viitorul imediat, glisezi fereastra. Variante: expanding window (toate datele anterioare) sau sliding window (fereastra fixa).

```
def walk_forward_validation(X, y, train_size=1000, test_size=100, step=100):
    results = []
    for i in range(0, len(X) - train_size - test_size, step):
        train_idx = slice(i, i + train_size)
        test_idx = slice(i + train_size, i + train_size + test_size)

        model = XGBRegressor()
        model.fit(X[train_idx], y[train_idx])
        score = model.score(X[test_idx], y[test_idx])
        results.append({'start': i, 'score': score})
    return pd.DataFrame(results)
```

5.2. Blocked CV cu gap (anti-leakage advanced)

Pentru serii cu autocorelatie reziduala, intre train si test pui un buffer (gap) ca sa eviti leakage prin autocorelatie.

```
def blocked_cv_with_gap(X, n_splits=5, gap=24):
    n = len(X)
    block_size = n // n_splits
    for i in range(n_splits - 1):
        train_end = (i + 1) * block_size
        test_start = train_end + gap
        test_end = test_start + block_size
        if test_end > n:
            break
        yield range(train_end), range(test_start, test_end)
```

5.3. Group Stratified K-Fold

Cand ai grupuri (pacientii) si clase dezechilibrate, vrei AMBELE: grupurile intregi in train sau test SI proportia claselor pastrata.

```
from sklearn.model_selection import StratifiedGroupKFold

sgkf = StratifiedGroupKFold(n_splits=5)
for train_idx, test_idx in sgkf.split(X, y, groups=patient_ids):
    # train si test au proportii similare de clase
    # si nicio persoana nu e in ambele
    pass
```

5.4. Conformal Prediction - matematica

Idea de baza: in loc sa prezici o valoare punctuala, prezici un interval cu garantie statistica de acoperire.

Algoritm split conformal: 1) Antrenezi modelul pe train. 2) Calculezi rezidualele pe un calibration set: $r_i = |y_i - \hat{y}_i|$. 3) Pentru o noua predictie $\hat{y}$, intervalul e $[\hat{y} - q_{(1-\alpha)}, \hat{y} + q_{(1-\alpha)}]$ unde q e cuantila reziduurilor.

Garantie matematica: $P(y \text{ in interval}) \geq 1 - \alpha$. De exemplu, pentru $\alpha = 0.05$, garantia e ca 95% din predictiile reale vor fi in interval.

```
from mapie.regression import MapieRegressor

mapie = MapieRegressor(model, method='plus')
mapie.fit(X_train, y_train)
y_pred, y_pis = mapie.predict(X_test, alpha=0.05)
# y_pis: prediction intervals [lower, upper]
```

Categoria 6 (avansat). Modele de Machine Learning

6.1. XGBoost - parametri avansati

Regularizare matematica

XGBoost minimizeaza: $Obj = \sum_i loss(y_i, \hat{y}_i) + \sum_t \Omega(f_t)$.

$\Omega(f) = \gamma * T + (1/2) * \lambda * ||w||^2 + \alpha * ||w||_1$.

T = numar frunze. γ = penalitate pe complexitate (controleaza pruning). λ = L2 regularization pe weights frunze. α = L1 regularization.

Hyperparametri principali (cu interpretare)

n_estimators (default 100): numar arbori. Mai mare = mai puternic dar overfitting. Optim: 200-2000 cu early stopping.

max_depth (default 6): adancime maxima. 3 = simplu, 8 = complex. Tipic 4-8.

learning_rate (default 0.3): cat de mult corecteaza fiecare arbore. Tipic 0.01-0.3. Mai mic = mai stabil dar mai lent.

subsample (default 1.0): fractiune din randuri folosite per arbore. 0.7-0.9 pentru regularizare.

colsample_bytree (default 1.0): fractiune din coloane per arbore. 0.7-0.9 pentru regularizare.

γ (default 0): minim loss reduction pentru split. Mai mare = mai conservator (mai putine splits).

λ , α (default 1, 0): regularizare L2, L1 pe weights frunze.

Setarile recomandate pentru proiecte tipice

Baseline: n_estimators=300, max_depth=6, learning_rate=0.1. Functioneaza in 80% din cazuri.

Cu tuning Optuna: search space n_estimators=[100, 1000], max_depth=[3, 10], learning_rate=[0.01, 0.3] in log scale, subsample=[0.6, 1.0], colsample_bytree=[0.6, 1.0].

6.2. LightGBM vs XGBoost - diferenta tehnica

Crestere arbori: XGBoost = level-wise (echilibrat). LightGBM = leaf-wise (preferential). Leaf-wise creste arborele in jurul frunzei cu cea mai mare imbunatatire potentiala.

Avantajul leaf-wise: mai rapid pentru date mari (skip multi noduri), accuracy mai buna in unele cazuri. Dezavantajul: poate overfita pe date mici (necesita max_depth strict).

Pentru date sub 10K randuri: XGBoost. Pentru date peste 100K: LightGBM. Pentru categoricals native: CatBoost.

6.3. LSTM - arhitectura detaliata

Celula LSTM are 4 componente: forget gate (f), input gate (i), cell state (C), output gate (o).

Forget gate: $f_t = \text{sigmoid}(W_f * [h_{(t-1)}, x_t] + b_f)$. Decide ce sa uite din cell state.

Input gate: $i_t = \text{sigmoid}(W_i * [h_{(t-1)}, x_t] + b_i)$. Decide ce informatie noua sa adauge.

Candidate values: $\tilde{C}_t = \tanh(W_C * [h_{(t-1)}, x_t] + b_C)$.

Cell state: $C_t = f_t * C_{(t-1)} + i_t * \tilde{C}_t$. Memoria pe termen lung.

Output gate: $o_t = \text{sigmoid}(W_o * [h_{(t-1)}, x_t] + b_o)$. Decide ce parte din memoria sa foloseasca.

Hidden state: $h_t = o_t * \tanh(C_t)$.

6.4. Bidirectional LSTM

Pentru secvente unde context-ul viitor conteaza (NLP, anumite time series cu vizibilitate completa), folosesti BiLSTM. Procezezi secventa in ambele directii si concatenezi outputs.

```
from tensorflow.keras.layers import Bidirectional, LSTM

model.add(Bidirectional(LSTM(64, return_sequences=True)))
```

Atentie: pentru forecasting clasic NU folosi BiLSTM - in productie nu ai acces la viitor. BiLSTM e pentru clasificare secvente sau encoders in seq2seq.

6.5. Transformer - matematica self-attention

Self-attention: $\text{Attention}(Q, K, V) = \text{softmax}(Q * K^T / \sqrt{d_k}) * V$.

Q (query), K (key), V (value) sunt obtinute prin transformari liniare ale input-ului: $Q = X * W_Q$, $K = X * W_K$, $V = X * W_V$.

$Q * K^T$: scoreaza compatibilitatea fiecarui token cu fiecare alt token. $\sqrt{d_k}$: scaling pentru stabilitate numerica. Softmax: transforma scor-uri in ponderi care suma la 1.

Multi-head attention: ruleaza N atentii in paralel cu Q, K, V proiectate diferit, apoi concatenezi. Permite invatare de tipuri diferite de relatii in paralel.

6.6. Positional Encoding

Transformers nu au notiune nativa de ordine (spre deosebire de RNN). Pentru a incorpora ordinea, adaugi positional encoding la embedding-uri.

Sinusoidal (Vaswani et al, 2017): $PE(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d)})$, $PE(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d)})$.

Avantaj: extrapoleaza la lungimi de secventa nevazute. Modern: rotary position embeddings (RoPE) folosite in LLaMA, mai performant pentru long context.

6.7. TFT (Temporal Fusion Transformer) - arhitectura

TFT combina LSTM (pentru pattern locale) + Multi-head attention (pentru pattern globale) + Variable Selection Networks (selectie automata features).

Avantaj decisiv: interpretabilitate - vezi care features si care timestamps influenteaza predictiile prin attention weights. State-of-the-art pe benchmarks de forecast multi-orient.

Categoria 7 (avansat). Hyperparameter Tuning

7.1. Optuna avansat - TPE algorithm

Optuna foloseste Tree Parzen Estimator (TPE) ca algoritm Bayesian default. TPE imparte trial-urile bune (top 25% din scoruri) de cele slabe si aproximeaza distributia buna ca $P(x | y < y^*)$. Apoi alege urmatorul x care maximizeaza $P(x | y < y^*) / P(x | y \geq y^*)$.

Avantaj fata de Gaussian Process: scaleaza bine la spatii high-dimensional (peste 30 hyperparametri). Functioneaza si pe spatii mixte (categorice + continue + integer).

7.2. Optuna pruners (early stopping pentru tuning)

```
import optuna

def objective(trial):
    params = {
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('lr', 0.01, 0.3, log=True),
    }
    model = XGBRegressor(**params)

    # Pruning: dupa fiecare iteratie de antrenare, raporteaza scor
    for step in range(10):
        model.fit(X_train_partial, y_train_partial)
        score = model.score(X_val, y_val)
        trial.report(score, step)
        if trial.should_prune():
            raise optuna.TrialPruned()
    return score

pruner = optuna.pruners.MedianPruner()
study = optuna.create_study(direction='maximize', pruner=pruner)
study.optimize(objective, n_trials=100)
```

MedianPruner: opreste trial-urile care performeaza sub mediana primelor n trials. Economisesti pana la 50% din timp.

7.3. Multi-objective optimization

Cand vrei sa optimizezi mai multe metrice simultan (de exemplu, accuracy si latency), folosesti optimizare multi-obiectiv.

```
def objective(trial):
    params = {...}
    model = ...

    accuracy = ...
    inference_time = measure_inference_time(model)

    return accuracy, inference_time # tuple

study = optuna.create_study(directions=['maximize', 'minimize'])
study.optimize(objective, n_trials=100)

# Pareto front: trial-urile care nu sunt dominate
pareto_trials = study.best_trials
```

7.4. Hyperband - matematica

Idea: testeaza N combinatii cu putin budget. Elimina cele slabe. Repeti cu mai mult budget pe cele bune. Repeti.

Bracket structure: Inceputul cu $N=81$ trials x 1 epoca, apoi 27×3 , apoi 9×9 , apoi 3×27 , apoi 1×81 . Total budget = $81 * 5 = 405$ epoci.

Cand functioneaza bine: cand performanta intermediara prezice performanta finala (deep learning). Cand nu: cand antrenarea e zgomotoasa (XGBoost cu shuffle).

Categoria 8 (avansat). Metrice de Evaluare

8.1. Calibration plots

Modelul tau spune 'probabilitate 0.8'. Daca te uiti la toate predictiile cu probabilitate ~0.8, ar trebui ca ~80% sa fie pozitive in realitate. Daca nu, modelul e miscalibrat.

```
from sklearn.calibration import calibration_curve
import matplotlib.pyplot as plt

prob_true, prob_pred = calibration_curve(y_true, y_prob, n_bins=10)
plt.plot(prob_pred, prob_true, marker='o')
plt.plot([0, 1], [0, 1], '--', color='gray')
plt.xlabel('Mean predicted probability')
plt.ylabel('Fraction of positives')
```

Linia punctata = calibrare perfecta. Punctele tale ar trebui sa fie aproape de aceasta linie.

8.2. Calibration techniques

Platt scaling: Antrenezi o regresie logistica peste probabilitatile modelului tau. Bun pentru SVM, modele care nu produc probabilitati native.

Isotonic regression: regresie monoton crescatoare. Mai flexibila dar necesita mai multe date pentru a evita overfitting.

```
from sklearn.calibration import CalibratedClassifierCV

cal_model = CalibratedClassifierCV(base_model, method='isotonic', cv=5)
cal_model.fit(X_train, y_train)
```

8.3. Brier Score - decompozitie

Brier Score = $\text{mean}((y_{\text{pred}} - y_{\text{true}})^2)$ pentru clasificare binara. Decompozitie Murphy: Brier = Reliability - Resolution + Uncertainty.

Reliability: cat de aproape sunt predictiile de frecventele observate (calibration).

Resolution: cat de bine separa modelul cazurile (discrimination).

Uncertainty: incertitudinea inerenta in date.

Pentru a imbunatati Brier Score: imbunatatesti calibration (recalibrezi) sau resolution (model mai bun).

8.4. Threshold optimization

Pragul default 0.5 pentru clasificare binara e arbitrar. Optim depinde de costurile false positives vs false negatives.

```
from sklearn.metrics import precision_recall_curve

precision, recall, thresholds = precision_recall_curve(y_true, y_proba)

# F1 optim
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-10)
best_threshold = thresholds[f1_scores.argmax()]

# Cost-sensitive
cost_fp = 1.0
```

```
cost_fn = 5.0 # FN costa de 5x mai mult
expected_cost = cost_fn * (1 - precision) + cost_fn * (1 - recall)
best_threshold = thresholds[expected_cost.argmin()]
```

8.5. Custom losses

Pentru probleme business, deseori metrica oficiala nu e cea mai potrivita. Construisti custom loss.

Exemplu finance: penalty asimetric. Subestimarea costa mai mult decat supraestimarea (sau invers).

```
def asymmetric_loss(y_true, y_pred, alpha=2.0):
    diff = y_pred - y_true
    return np.where(diff > 0, alpha * diff**2, diff**2).mean()

# In XGBoost:
def custom_obj(y_true, y_pred):
    grad = ... # gradientul lossului
    hess = ... # hessianul
    return grad, hess
```

Categoria 9 (avansat). Explainability

9.1. SHAP - matematica completa

Shapley value pentru feature i: $\phi_i = \sum_{S \subset N \setminus \{i\}} (|S|! * (n - |S| - 1)! / n! * (v(S \cup \{i\}) - v(S)))$.

Intuitiv: pentru fiecare 'coalitie' S de features, calculezi cat aduce in plus feature i daca e adaugat la S. Mediezi peste toate coalitions posibile (cu ponderi care depind de marimea coalition-ului).

Proprietati matematice: Efficiency (suma SHAP values = predictia - baseline), Symmetry (features identice primesc valori egale), Dummy (feature inutil primeste 0), Additivity (combinatii de modele = suma SHAP values).

9.2. SHAP variants

TreeExplainer: pentru tree-based models. Calcul exact in $O(TLD^2)$ unde T=arbori, L=frunze, D=adancime.

DeepExplainer: pentru retele neurale. Aproximatie folosind gradient * input.

KernelExplainer: model-agnostic. Lent dar functioneaza pe orice model. Foloseste sampling.

LinearExplainer: pentru modele liniare. Coeficienti scalati.

9.3. SHAP Interaction values

Pentru a vedea interactiunile intre features, calculezi SHAP interaction values: matrix unde ϕ_{ij} = contributia perechii (feature i, feature j).

```
import shap

explainer = shap.TreeExplainer(model)
interaction_values = explainer.shap_interaction_values(X)

# interaction_values are shape (n_samples, n_features, n_features)
# Diagonala = main effect, off-diagonal = interactiuni
```

Util pentru detectia features care lucreaza impreuna. De exemplu, in healthcare: efectul varstei depinde de gen.

9.4. Counterfactual explanations cu DiCE

Pentru o predictie data, gaseste cele mai mici modificari care ar schimba predictia. Foarte util pentru recomandari catre utilizatori.

```
import dice_ml

d = dice_ml.Data(dataframe=df, continuous_features=['age', 'income'], outcome_name='target')
m = dice_ml.Model(model=model, backend='sklearn')
exp = dice_ml.Dice(d, m)

counterfactuals = exp.generate_counterfactuals(
    query_instance, total_CFs=5, desired_class='opposite'
)
counterfactuals.visualize_as_dataframe()
```

Aplicatie: 'Cererea de credit a fost respinsa. Daca ai avea income >5000 EUR si scor credit >700, ar fi aprobata.'

9.5. ALE (Accumulated Local Effects)

Alternativa moderna la PDP. PDP presupune independenta features, ALE este robust la corelari.

Algoritm: 1) Imparte range-ul feature-ului in K bins. 2) Pentru fiecare bin, calculezi differential effect (diferenta intre predictia la marginea de sus si cea de jos a bin-ului). 3) Acumulezi diferentele pentru a obtine curva ALE.

```
from alibi.explainers import ALE

ale = ALE(model.predict, feature_names=X.columns)
ale_exp = ale.explain(X.values, features=[0])
ale_exp.plot()
```

Categoria 10 (avansat). Deployment

10.1. Docker pentru ML - best practices

Multi-stage build: imagine de build separata de imagine de runtime. Reduci dimensiunea finala cu 50-80%.

```
# Stage 1: Builder
FROM python:3.11 AS builder
COPY requirements.txt .
RUN pip install --user -r requirements.txt

# Stage 2: Runtime
FROM python:3.11-slim
COPY --from=builder /root/.local /root/.local
COPY ./app /app
WORKDIR /app
CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

10.2. FastAPI cu validare Pydantic

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
from typing import List
import joblib

app = FastAPI(title='ML Model API', version='1.0')
model = joblib.load('model.pkl')

class PredictRequest(BaseModel):
    features: List[float] = Field(..., min_items=10, max_items=10)

class PredictResponse(BaseModel):
    prediction: float
    confidence: float

@app.post('/predict', response_model=PredictResponse)
def predict(request: PredictRequest):
    pred = model.predict([request.features])[0]
    proba = model.predict_proba([request.features])[0].max()
    return PredictResponse(prediction=pred, confidence=proba)
```

Pydantic valideaza automat input-ul. Daca request-ul nu corespunde schemei, FastAPI returneaza 422 cu detalii.

10.3. ONNX optimization

```
import onnx
from onnxruntime.quantization import quantize_dynamic, QuantType

# Quantization: float32 -> int8
quantize_dynamic(
    'model.onnx',
    'model_quantized.onnx',
    weight_type=QuantType.QInt8
)

# Inferenta cu ONNX Runtime
import onnxruntime as ort
session = ort.InferenceSession('model_quantized.onnx')
outputs = session.run(None, {'input': X.astype(np.float32)})
```


Quantization reduce dimensiunea modelului de 4x si accelereaza inference de 2-3x. Ideala pentru deployment edge (mobile, IoT).

10.4. Model versioning cu MLflow

```
import mlflow

mlflow.set_experiment('production_model')

with mlflow.start_run():
    mlflow.log_params(params)
    mlflow.log_metrics(metrics)
    mlflow.sklearn.log_model(model, 'model', registered_model_name='disertatie_xgboost')

# Promote la productie
client = mlflow.MlflowClient()
client.transition_model_version_stage(
    name='disertatie_xgboost',
    version=5,
    stage='Production'
)
```

10.5. Blue/Green deployment

Doua medii identice: Blue (productie curenta) si Green (versiune noua). Routezi 100% traffic la Blue. Cand Green e validat, comuti router-ul. Daca apar probleme, comuti inapoi instant.

Avantaj: zero downtime, rollback instant. Implementat cu load balancer (nginx, AWS ALB) sau service mesh (Istio).

Categoria 11 (avansat). Monitoring

11.1. PSI calculation manual

PSI (Population Stability Index) = $\sum_i (\text{actual_pct}_i - \text{expected_pct}_i) * \log(\text{actual_pct}_i / \text{expected_pct}_i)$.

Interpretare: PSI < 0.1 = no significant change. 0.1 <= PSI < 0.25 = moderate change, investigate. PSI >= 0.25 = significant drift, retrain model.

```
def calculate_psi(expected, actual, n_bins=10):
    bins = np.linspace(min(expected.min(), actual.min()),
                        max(expected.max(), actual.max()), n_bins+1)

    expected_pct = np.histogram(expected, bins=bins)[0] / len(expected)
    actual_pct = np.histogram(actual, bins=bins)[0] / len(actual)

    expected_pct = np.where(expected_pct == 0, 1e-6, expected_pct)
    actual_pct = np.where(actual_pct == 0, 1e-6, actual_pct)

    psi = np.sum((actual_pct - expected_pct) * np.log(actual_pct / expected_pct))
    return psi
```

11.2. KS test pentru drift

```
from scipy.stats import ks_2samp

stat, p_value = ks_2samp(reference_data, current_data)
if p_value < 0.05:
    print('Drift detected!')
```

KS test compara doua distributii. Detecteaza orice diferenta in distributie (mean, variance, shape). Mai sensibil decat PSI.

11.3. Population shift vs concept drift

Population shift: distributia features X se schimba ($P(X)$ se modifica). Exemplu: alti utilizatori vin la site dupa o campanie.

Concept drift: relatia $X \rightarrow y$ se schimba ($P(y|X)$ se modifica). Exemplu: comportamentul cumparatorilor s-a schimbat dupa pandemie.

Diagnostic: pentru population shift, monitor PSI pe features. Pentru concept drift, monitor performance metrics (RMSE, accuracy) cu ground truth labels intarziate.

11.4. Prometheus + Grafana

```
from prometheus_client import Counter, Histogram, start_http_server

prediction_counter = Counter('ml_predictions_total', 'Total predictions made')
latency_histogram = Histogram('ml_prediction_latency_seconds', 'Prediction latency')

@latency_histogram.time()
def predict(features):
    prediction_counter.inc()
    return model.predict(features)

start_http_server(8000) # /metrics endpoint
```

Prometheus scrapeaza metricele tale la /metrics endpoint. Grafana le vizualizeaza in dashboard-uri. Standard pentru observability in productie ML.

11.5. Alerting strategies

Alerts pe drift: $PSI > 0.25$ trigger warning, $PSI > 0.5$ trigger critical.

Alerts pe performance: dropping accuracy below 0.9 baseline, increased error rate.

Alerts pe latency: P95 latency $> 100ms$ (varies pe SLO).

Alerts pe data quality: missing values increase, schema changes.

Categoria 12 (avansat). Tehnici Emergente

12.1. RAG architecture detaliata

Componente RAG: 1) Document Loader (PDF, HTML, Markdown). 2) Text Splitter (chunks de 500-1000 tokens cu overlap). 3) Embedding Model (text-embedding-3-small, BGE, sentence-transformers). 4) Vector Database (Pinecone, Weaviate, Chroma). 5) Retriever (similarity search). 6) Reranker (optional, imbunatateste relevanta). 7) Generator LLM (GPT-4, Claude, Llama).

Workflow: User query -> Embed query -> Search vector DB -> Top-K documents -> Rerank -> Build context -> LLM generates answer.

12.2. Vector embeddings - matematica

Embedding: $f(\text{text}) \rightarrow \mathbb{R}^d$, unde d e dimensiunea (384, 768, 1536, 3072 tipic). Texte similare semantic sunt apropiate in spatiul vectorial.

Cosine similarity: $\cos(\theta) = (A \cdot B) / (||A|| * ||B||)$. Standard pentru similarity in NLP. Range $[-1, 1]$, practic $[0, 1]$ dupa normalizare.

Modele moderne (2025): text-embedding-3-large (OpenAI, 3072 dim), BGE-M3 (multilingual, 1024 dim), E5 (Microsoft, varianta multiple).

12.3. ANN search algorithms

Pentru a cauta similar in milioane de vectori, foloseste Approximate Nearest Neighbor algorithms. Trade-off: viteza vs accuracy.

HNSW (Hierarchical Navigable Small World): graf cu multiple straturi. Cautare in $O(\log N)$. Cel mai folosit, default in majoritatea vector DB.

IVF (Inverted File Index): clustering + cautare doar in cluster relevant. Mai rapid la indexing, mai lent la query.

PQ (Product Quantization): comprima vectori in coduri scurte. Reduce memorie cu 90% cu pierdere mica de accuracy.

12.4. LoRA mathematics

LoRA (Low-Rank Adaptation): in loc sa fine-tunezi toate parametrii W din retea, decompoti $\Delta W = A * B$ unde A si B sunt matrici low-rank (rank $r=8$ tipic).

Matematic: $W' = W + \alpha * A * B$, unde W e original (frozen), A si B sunt invatabile (mici).

Reducere parametri: pentru un layer cu $1024 \times 1024 = 1M$ parametri, LoRA cu $r=8$ are doar $1024 * 8 + 8 * 1024 = 16K$ parametri (60x reducere).

Avantaj: poti fine-tuna LLM-uri mari (Llama-7B) pe consumer GPU (24GB VRAM).

12.5. QLoRA quantization

QLoRA = LoRA + quantization 4-bit. Modelul de baza e cuantizat la 4-bit (NF4 - Normal Float 4) ceea ce reduce memoria de 8x. LoRA adaptarile raman in 16-bit pentru gradient stability.

Cu QLoRA poti fine-tuna Llama-65B pe un GPU 48GB. Permite democratizarea fine-tuning-ului LLM-urilor mari.

12.6. Reranking

Dupa retrieving top-K documents prin similarity, foloseste un model mai puternic (cross-encoder) pentru a re-scora si re-ordona.

Cross-encoder: ia (query, document) ca input si returneaza score. Mai lent decat bi-encoder dar mult mai precis.

Modele populare: ms-marco-MiniLM, BGE-reranker, Cohere Rerank API. Imbunatateste relevanta cu 10-30%.

12.7. Function calling protocols

Pattern: LLM primeste schema JSON cu functiile disponibile. La un query, LLM decide daca sa apeleze o functie si returneaza arguments. Tu executi functia si dai rezultatul inapoi LLM-ului.

Standard: OpenAI function calling, Anthropic tool use. Foundation pentru agents AI moderne.

```
tools = [{
    'name': 'get_weather',
    'description': 'Get current weather for a city',
    'parameters': {
        'type': 'object',
        'properties': {
            'city': {'type': 'string', 'description': 'City name'}
        },
        'required': ['city']
    }
}]

response = client.chat.completions.create(
    model='gpt-4',
    messages=[{'role': 'user', 'content': 'Weather in Bucharest?'}],
    tools=tools
)
```

12.8. Fine-tuning pipeline complet

Pasi: 1) Colectare data (instruction-following format). 2) Cleaning si formatare (chat templates). 3) Train/val split. 4) Setup LoRA / QLoRA. 5) Antrenare cu Hugging Face TRL (SFTTrainer). 6) Evaluation pe holdout set. 7) Merge LoRA inapoi in model. 8) Deploy.

```
from trl import SFTTrainer
from peft import LoraConfig

lora_config = LoraConfig(r=8, lora_alpha=16, target_modules=['q_proj', 'v_proj'])

trainer = SFTTrainer(
    model=base_model,
    train_dataset=train_data,
    peft_config=lora_config,
    max_seq_length=2048,
)
trainer.train()
```

Roadmap pentru a deveni expert ML

Faza 1: Fundamente (1-3 luni)

Algebra liniara, calcul, statistica de baza. Python (pandas, numpy, sklearn). Primul proiect end-to-end. Lectura: Hands-On Machine Learning (Aurelien Geron).

Faza 2: ML clasic (3-6 luni)

Tree-based models in profunzime (XGBoost, LightGBM). Feature engineering practic (Kaggle). SHAP si interpretabilitate. Time series fundamentale. Lectura: An Introduction to Statistical Learning.

Faza 3: Deep Learning (6-9 luni)

PyTorch sau TensorFlow. CNN pentru imagini, Transformers pentru NLP. Fine-tuning si transfer learning. Hugging Face ecosystem. Lectura: Deep Learning (Goodfellow), course-uri fast.ai.

Faza 4: MLOps si productie (9-12 luni)

Docker, FastAPI, MLflow. Cloud (AWS / GCP / Azure). Monitoring si alerting. Cursuri: Made With ML, Full Stack Deep Learning.

Faza 5: Specializare (12+ luni)

Alegi un domeniu: NLP avansat, Computer Vision, Time Series, Recommender Systems, Causal ML. Citesc papers din 2-3 conferinte top (NeurIPS, ICML, ICLR). Contribui la open source. Mentorship invers - explici altora pentru a consolida.

Resurse recomandate de invatare

Carti: Hands-On Machine Learning (Geron), Deep Learning (Goodfellow), An Introduction to Statistical Learning (James et al), Designing Machine Learning Systems (Huyen).

Cursuri: Andrew Ng (Coursera), fast.ai, deeplearning.ai. Cursuri specializate: Hugging Face NLP Course, Made With ML.

Practica: Kaggle (competitii cu mentorat de la top kagglers). Open source contributions. Personal portfolio cu 3-5 proiecte end-to-end.

Comunitati: r/MachineLearning, Hacker News, Twitter ML community, conferinte (PyData, NeurIPS, ICML).

Newsletter: The Batch (Andrew Ng), Import AI, Deeplearning.ai newsletter, Towards Data Science (Medium).

Concluzie

Devenirea expert in ML nu e o destinatie, ci un proces continuu. Tehnologia evolueaza rapid - ce e state-of-the-art azi devine baseline maine. Cheia e sa intelegi fundamentele matematice si principiile (de ce functioneaza X) atat de bine, incat sa poti adopta rapid orice tehnica noua.

Acest manual contine fundamentele si tehnicile actuale. Recitirea periodica si aplicarea practica in proiecte vor consolida cunostintele. La fiecare 6 luni, recitesti pentru a vedea ce s-a schimbat in domeniu si pentru a-ti reconfirma cunostintele de baza.

Mult succes in cariera ta de Data Scientist / ML Engineer.